Hostel Visitor Management System

End-to-End Cloud Solution Design and Deployment on Amazon Web Services

Programme	Azubi Africa, Portfolio Project 1
Prepared by	Documentation Lead
Project	End-to-End Cloud Solution Design and Deployment
Application	UPSA Hostel Visitor Management System
GitHub repository	Ewura-bena685/Lambda_Capstone_aws-secure-web-app
Report date	4 November 2026
Version	2.0. Combines the Setup Documentation (v1.1, 31 October 2026) and the Implementation Report
Status	Deliverables Completed
Cloud region	Europe (Stockholm), eu-north-1

Contents

1. Executive Summary 2
Project Implementation and Completion Status 2
2. Introduction and problem statement 3
2.1 The problem 3
2.2 What the application solves 3
2.3 The technical problem and the cloud solution 3
2.4 Scope of this document 4
3. The application 4
3.1 Functions 4
3.2 Technology stack 4
3.3 Why the application shapes the architecture 5
3. Solution architecture 5
4.1 Request flow 5
4.2 Why each service was chosen 6
4.3 Design decisions and trade-offs 6
5. Phase 1: Foundation and source control 7
5.1 AWS identity 7
5.2 Source control 7
5.3 Project management 8
5.4 Database 9
6. Phase 2: Compute, load balancing and application deployment 9
6.1 Network security: security groups 9
6.2 IAM role for the server 10
6.3 EC2 instance 12
6.4 Server configuration and build 12
6.5 Running the application as a service 14
6.6 Target group 14
6.7 Application Load Balancer 15
6.8 Functional check through the load balancer 16
7. Phase 3: Static content delivery and network lockdown 16
7.1 S3 bucket for static files 16
7.2 CloudFront distribution 17
7.3 Locking down the load balancer 19
8. Phase 4: Monitoring, cost control and continuous deployment 19
8.1 Notification channel 19
8.2 CloudWatch alarms 20
8.3 Budget 21
8.4 Application logs 21
8.5 Continuous deployment with GitHub Actions 22
9. Security design 24
10. Cost management 24
11. Verification results 25
12. Troubleshooting record 25
Challenge 1: Dependency conflict during installation 25
Challenge 2: Network time-out while downloading packages 26
Challenge 3: Build failed on a TypeScript error 27
Challenge 4: Push to the team repository rejected (403) 27
Challenge 5: EC2 console Connect button failed 28
Challenge 6: Security group rule could not be edited 28

Challenge 7: Service failed to start because the port was in use 29
Challenge 8: Load balancer returned 503 30
13. Lessons Learned 30
14. Repository contents 31
Appendix A: Resource inventory 32
Appendix B: Configuration listings 32

1. Executive Summary

The Hostel Visitor Management System is a cloud-based web application developed to simplify and improve hostel visitor management and package tracking. The system provides essential functionalities such as visitor registration, check-in and check-out, visitor record management, dashboard monitoring, and package registration and tracking.

The project was designed and deployed using Next.js, Supabase and Amazon Web Services (AWS). AWS services, including EC2, Application Load Balancer, Amazon S3 and CloudFront, provide the infrastructure for application hosting, traffic management and content delivery. Supabase serves as the database for storing visitor and package records.

To streamline deployment, GitHub Actions was integrated to automate application updates, while Trello was used to coordinate project activities and track implementation milestones. The successful GitHub Actions workflow and the running application on EC2 demonstrate the achievement of key deployment objectives.

Overall, the project demonstrates practical experience in cloud infrastructure deployment, CI/CD automation, database integration, application hosting and IT operations. Further improvements will focus on strengthening application security, completing monitoring configurations and enhancing scalability to support future growth

1.1 Project Implementation and Completion Status

Area	Status	Notes
Identity and access management (IAM)	Completed	IAM users, groups and roles configured. MFA enabled for the five team users.
GitHub OIDC authentication	Completed	Secure GitHub-to-AWS authentication configured and successfully tested.
GitHub Actions deployment workflow	Completed	Automated deployment workflow implemented and successfully tested, including application deployment and static asset synchronisation.
EC2, Nginx and Next.js hosting	Completed	Application hosted on EC2, managed through systemd and verified after reboot.
AWS Systems Manager Parameter Store	Completed	Three parameters configured and retrieved through the EC2 instance role.
Application Load Balancer and Auto Scaling	Completed	Infrastructure configured across two Availability Zones, with two healthy targets and no unhealthy instances.
Private S3 and CloudFront integration	Completed	Static assets uploaded to private S3 storage and successfully delivered through CloudFront.
HTTPS from CloudFront to ALB	Completed	Secure communication configured between CloudFront and the application origin.
Application functional testing	Completed	visitor check-in,check-out,package management and appointment workflows tested successfully.
Instance failure and recovery testing	Completed	Instance failure simulation performed to verify load balancing and automatic recovery.
CloudWatch and SNS monitoring	Completed	CPU and ALB 5xx alarms configured, with SNS email notifications tested.
AWS Budgets	Completed	Monthly budget configured with 80% actual-spending and 100% forecast-spending alerts.

2. Introduction and problem statement

2.1 The problem

Hostels and similar shared residences admit a continuous flow of outside people: friends and relatives of residents, delivery riders, repair workers and couriers. In many such places this traffic is recorded in a paper logbook, or not recorded at all. This creates four practical problems.

- **No reliable record of who is inside the building.** If an incident occurs, management cannot quickly establish who visited, whom they visited, when they arrived and whether they have left.
- **Weak security and accountability.** Visitors who are never checked out remain unaccounted for, and handwritten entries are easy to omit, alter or misread.
- **Lost or disputed packages.** Parcels left at reception are not logged against a resident, residents are not told that something has arrived, and there is no proof of who collected it.
- **No overview for management.** Totals such as the number of visitors currently on the premises or the number of uncollected packages cannot be seen without counting pages by hand.

2.2 What the application solves

The Hostel Visitor Management System (VMS) replaces the logbook with a web application that front-desk staff use from any browser. It addresses each problem above:

Problem	How the application addresses it
No record of visitors	Every visitor is captured at check-in with name, phone number, host, room, purpose of visit and a time stamp, and stored in a database.
Visitors not accounted for	The check-out page lists only visitors who are still inside. Staff close each visit with one action, which records the exit time.
Lost or disputed packages	Packages are logged against a resident, the resident can be marked as notified, and collection is recorded with a time stamp.
No overview for management	A dashboard summarises current and past visits from the live database.

2.3 The technical problem and the cloud solution

A working application is not sufficient on its own. The project brief describes a start-up whose web application suffered from slow page loads, inconsistent availability during peak traffic, concerns about security, a deployment process with no structure, and difficulty in collaboration between developers. This project answers those concerns by deploying the application on AWS with the following properties:

- **Performance:** static files are served from Amazon CloudFront edge locations, which are close to users, and are cached.
- **Availability:** an Application Load Balancer performs health checks and routes traffic only to a healthy server, and the application restarts automatically if it stops.
- **Security:** traffic is encrypted with HTTPS, each layer accepts traffic only from the layer in front of it, and the server holds no stored AWS access keys.

- **Structured delivery:** code is held in a shared GitHub repository, changes are reviewed through pull requests, and work is tracked on a Trello board.
- **Cost control:** small instance sizes, alerts on spending, and monitoring alarms that report problems by email.

2.4 Scope of this document

This document provides an overview of the design, development, implementation, testing, and deployment of the Hostel Visitor Management System. It explains the technologies and AWS services used, why they were selected, the challenges encountered and resolved, and where the project code and configuration files are stored in the repository. All project sections and implementation activities have been completed, with supporting screenshots included as evidence of the work carried out and the successful testing of the system.

3. The application

3.1 Functions

Function	Description
Visitor check-in	Staff enter the visitor's name and phone number, the host being visited, the room and the purpose of the visit. The system stores the entry with the arrival time.
Visitor check-out	The page lists visitors who are currently inside. Selecting Check out records the departure time.
Dashboard	Summary of current and past visits.
Packages	Staff register a package for a resident, mark the resident as notified, and record when it is collected.
Resident package view	A page where a resident can see the packages waiting for them.

3.2 Technology stack

Layer	Technology and reason
Framework	Next.js 16.4.4 with the App Router. It provides pages and server-side API routes in a single project.
Language and interface	TypeScript for type safety, React 18 for the interface and Tailwind CSS for styling.
Database	Supabase, a managed PostgreSQL service, with the tables visits and packages. A managed database removes the need to install, patch and back up a database server.
Server runtime	Node.js on Amazon Linux 2023, run as a systemd service.
Collaboration	Git and GitHub for version control; Trello for task tracking.

3.3 Why the application shapes the architecture

The application has server-side API routes (for example /api/visits and /api/packages/[id]/notify) that run code on a server and talk to the database using a secret key. Such an application cannot be hosted as plain static files on Amazon S3. It needs a running Node.js process. For this reason the application runs on an Amazon EC2 instance behind a load balancer, while S3 and CloudFront are used only for the static files that Next.js generates (JavaScript and CSS under /_next/static/).

3. Solution architecture

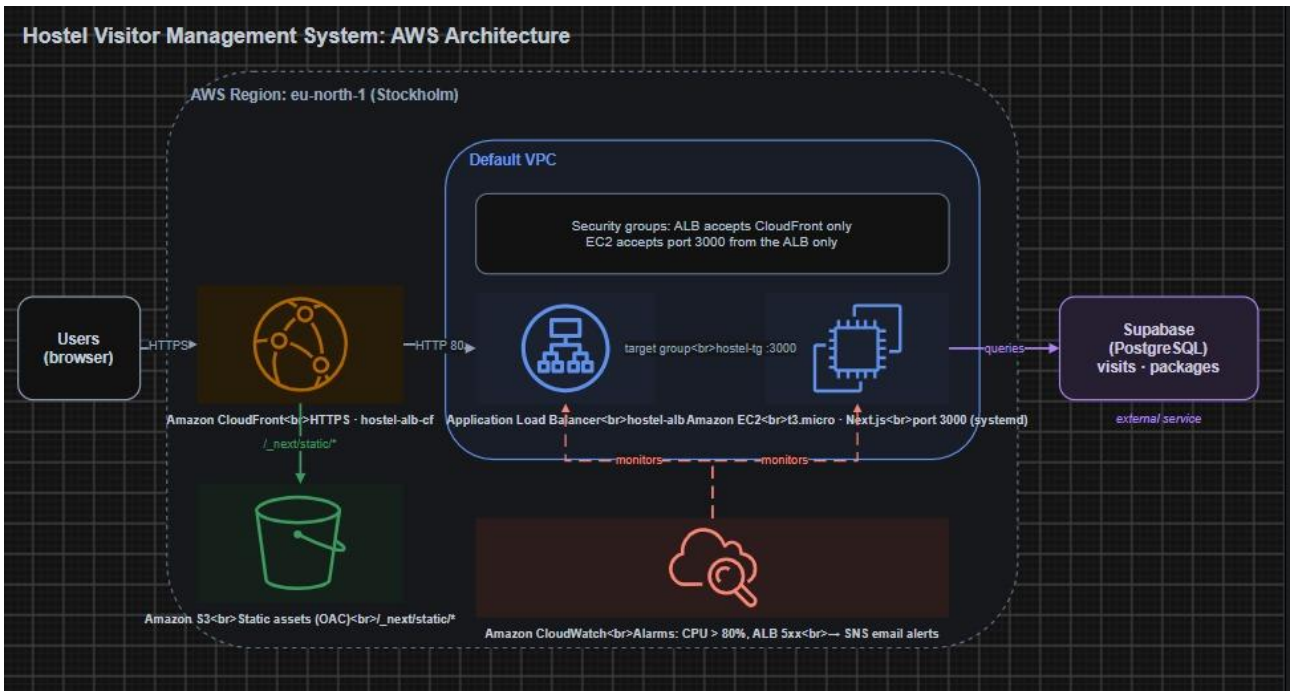


Figure 1. *Architecture of the Hostel Visitor Management System on AWS.*

What Figure 1 shows. The diagram shows the complete request path. Users connect to Amazon CloudFront over HTTPS. CloudFront has two origins: the S3 bucket, used for static files under /_next/static/, and the Application Load Balancer, used for everything else. Inside the default VPC, the load balancer forwards requests to the EC2 instance, which runs the Next.js server on port 3000. The application reads and writes data in Supabase, which is an external service outside AWS. Amazon CloudWatch monitors the load balancer and the instance and sends alerts through an SNS email topic. The box at the top of the VPC summarises the two security-group rules that restrict who can talk to whom.

4.1 Request flow

1. A user opens the CloudFront address. If the request uses HTTP, CloudFront redirects it to HTTPS.
2. If the path begins with /_next/static/, CloudFront returns the file from the S3 bucket (or from its edge cache).
3. Any other request, such as a page or an API call, is forwarded to the Application Load Balancer.

4. The load balancer selects a healthy target in the target group and forwards the request to the EC2 instance on port 3000.
5. The Next.js server processes the request and reads or writes data in Supabase over HTTPS, then returns the response along the same path.

4.2 Why each service was chosen

Service	Justification
Amazon EC2	Runs the Node.js server that the API routes require. A t3.micro instance is small and inexpensive, and is sufficient for a staff-facing application with modest traffic.
Application Load Balancer	Provides a single stable entry point, performs health checks and sends traffic only to healthy targets. It operates at the HTTP level, so it can be extended later with additional servers and path-based rules.
Target group	The list of servers the load balancer sends traffic to. It defines the port (3000) and the health-check path (/).
Amazon S3	Durable, low-cost storage for the static files. Keeping them in S3 reduces the load on the server.
Amazon CloudFront	Provides HTTPS without a custom domain, caches static files at edge locations for speed, and lets the load balancer be hidden from the public internet.
Security groups	Virtual firewalls that allow only the specific traffic each layer needs.
IAM role for EC2	Gives the server temporary permissions to AWS services without storing access keys on it.
Amazon CloudWatch and SNS	Detect high CPU use and server errors, and notify the administrator by email.
AWS Budgets	Warns as soon as spending appears so that cost does not grow unnoticed.
Supabase	Managed PostgreSQL database, separate from the server, so application data does not depend on a single EC2 instance.

4.3 Design decisions and trade-offs

- **HTTPS ends at CloudFront.** The project brief mentions an ACM certificate for a custom domain. No custom domain was used, so the default CloudFront certificate secures the public connection. The hop from CloudFront to the load balancer uses HTTP inside the AWS network, and the load balancer rejects any traffic that does not come from CloudFront. Adding a domain and certificate is listed as a future improvement.
- **One application server.** A single instance keeps cost low. The trade-off is that an instance failure causes downtime. An Auto Scaling group across Availability Zones is the recommended next step
- **Manual first deployment, followed by automated updates.** The initial deployment was completed manually to ensure that each step was properly understood and verified. Automated deployment was subsequently configured and tested using GitHub Actions, enabling future application updates to be deployed through the CI/CD pipeline (see Section 7.5).

.

5. Phase 1: Foundation and source control

5.1 AWS identity

All work is done with an IAM user instead of the root account. The root account has unrestricted power, including closing the account, so it should be used rarely and should be protected with multi-factor authentication. The EC2 server does not use access keys at all; it receives permissions through an IAM role (section 5.1).

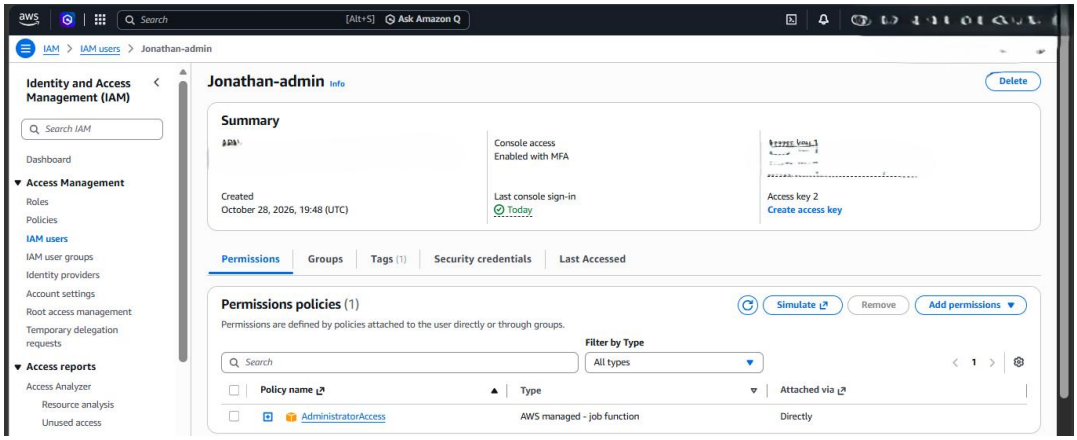


Figure 3. *Jonathan-admin Permissions tab showing AdministratorAccess attached.*

5.2 Source control

The code is kept in a team GitHub repository so that every member works from the same version, every change is recorded, and changes can be reviewed before they are merged. The web application lives in the src/ folder, so the server and the deployment workflow know exactly where to build it.

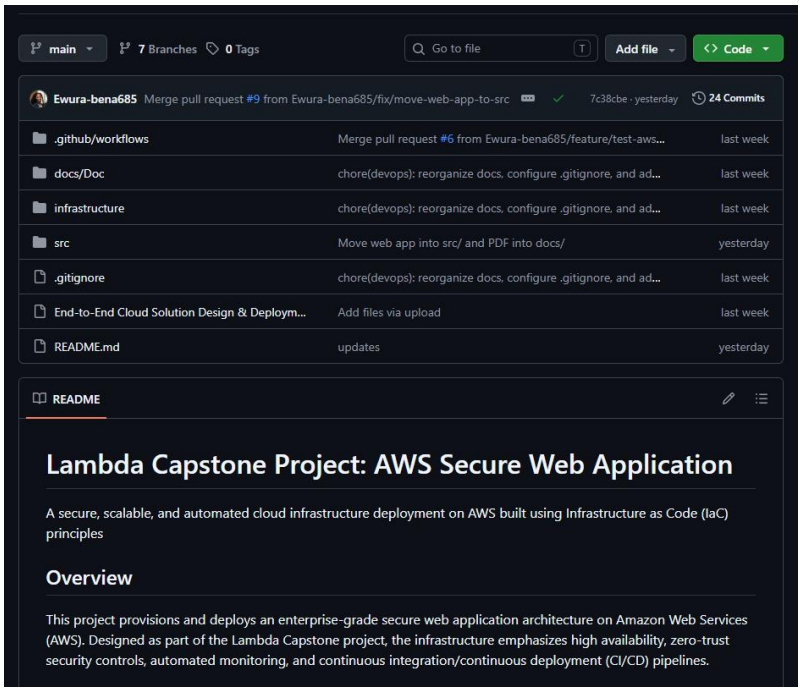


Figure 3. *Team GitHub repository, main branch.*

What Figure 3 shows. The repository page shows the main branch with 7 branches and 24 commits. The last commit is the merge of pull request #9 (fix/move-web-app-to-src), which is evidence that changes reach main through pull requests. The folder list shows docs for documents, src for the web application, and a .gitignore file that keeps secret files such as .env out of version control. The

README at the bottom still carries the earlier project title and is replaced by the README written for this project

```
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ git push
Username for 'https://github.com': dorcassai-ux
Password for 'https://dorcassai-ux@github.com':
Enumerating objects: 34, done.
Counting objects: 100% (34/34), done.
Delta compression using up to 4 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (18/18), 1.44 KiB | 210.00 KiB/s, done.
Total 18 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 7 local objects.
To https://github.com/Ewura-bena685/Lambda_Capstone_aws-secure-web-app.git
   7c38cbe..d5c1ca9  main -> main
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$
```

Figure 4. *Pushing the build fixes to the team repository.*

What Figure 4 shows. The terminal output shows git pushing 18 objects to the team repository, from commit 7c38cbe to d5c1ca9 on the main branch. This push contained the corrections needed to build the application with the installed version of Next.js: the .npmrc file and the three API route files described in section 11 (challenges 1 and 3). Pushing these changes matters because the EC2 server obtains its code from GitHub, so a fix that stays on a personal machine would not reach the server.

5.3 Project management

A Trello board with the columns Backlog, In Progress, Review and Done tracks the work, with one card per project phase. This gives the team a shared view of progress and fulfils the collaboration requirement of the project.

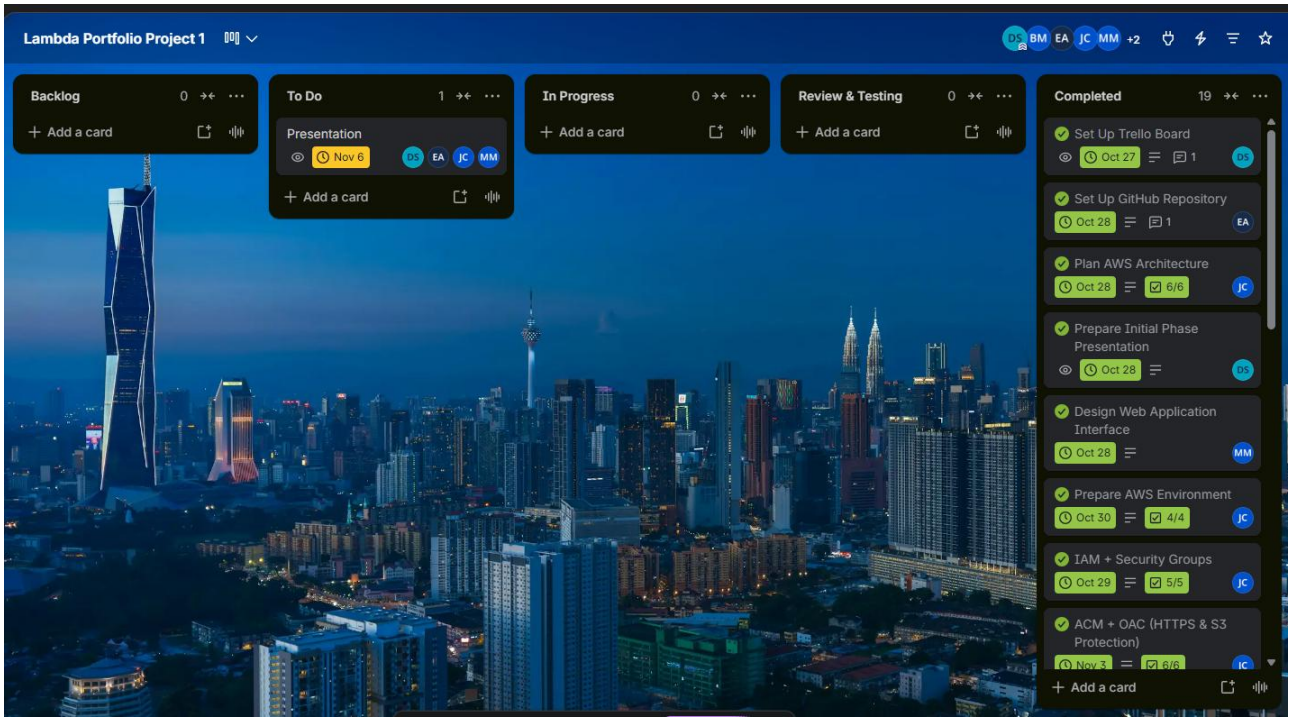


Figure 5. *Trello showing task progress for team members*

5.4 Database

The database is a Supabase project. Its tables, visits and packages, were created by running the file supabase-schema.sql in the Supabase SQL editor. The application reads three environment variables (the project URL, the public key and the service key). Keeping these in a .env file on the server, and excluding that file from Git, ensures no credentials appear in the repository. The application was first tested locally against this database before any AWS work began, so that any later failure could be attributed to AWS configuration and not to the application.

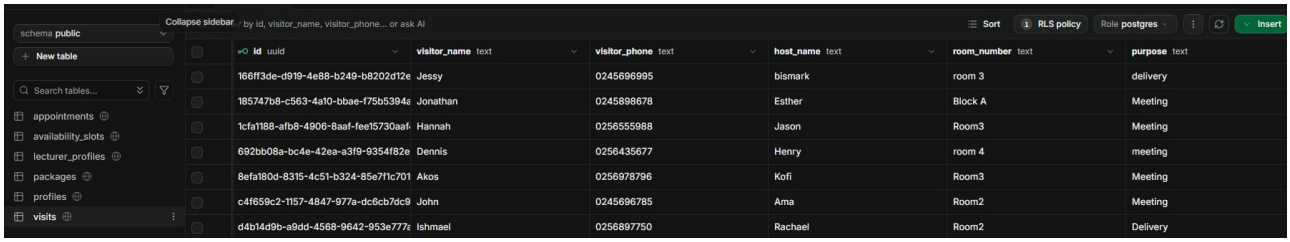


Figure 6 . *Supabase tables*

6. Phase 2: Compute, load balancing and application deployment

6.1 Network security: security groups

A security group acts as a firewall around a resource. Two groups were created so that each layer accepts only what it needs. The EC2 group refers to the load balancer's group by its identifier rather than by an IP address. As a result, only resources that belong to the load balancer's group can reach the application port, regardless of how addresses change.

Security group	Rule	Source	Reason
hostel-alb-sg	HTTP 80	CloudFront prefix list (final state)	Only CloudFront may reach the load balancer
hostel-ec2-sg	TCP 3000	hostel-alb-sg	Only the load balancer may reach the application
hostel-ec2-sg	SSH 22	Administrator's IP address	Remote administration from one known address

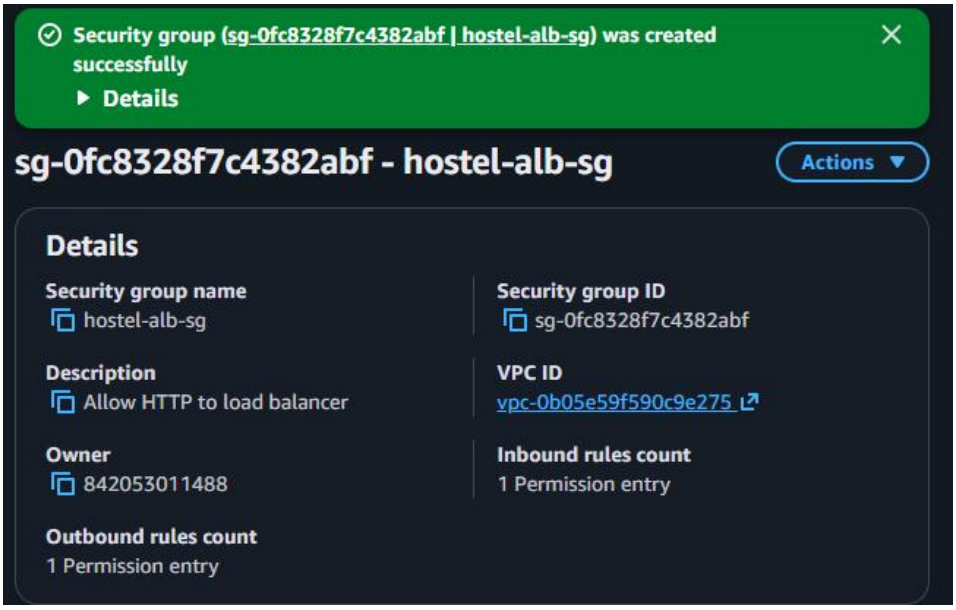


Figure 7. *Security group hostel-alb-sg created.*

What Figure 7 shows. The confirmation banner shows that the security group sg-0fc8328f7c4382abf, named hostel-alb-sg, was created. The details panel gives its description (Allow HTTP to load balancer), the default VPC it belongs to, one inbound rule (HTTP on port 80, which at creation allowed any address) and one outbound rule (the default, which allows all outgoing traffic so the load balancer can reach the server). The inbound rule was later tightened to the CloudFront prefix list (section 6.3). This group was created first because the EC2 group needs to refer to it.

6.2 IAM role for the server

The EC2 instance needs to call AWS services, for example to upload files to S3 and to send monitoring data. Storing access keys on a server is risky because anyone who reads the keys can use them from anywhere. An IAM role avoids this: AWS supplies the instance with temporary credentials that are renewed automatically and can only be used from that instance. The role hostel-ec2-role has three policies.

Policy	Reason
AmazonSSMManagedInstanceCore (AWS managed)	Allows AWS Systems Manager to manage the instance, which is used by the planned automatic deployment.
CloudWatchAgentServerPolicy (AWS managed)	Allows the CloudWatch agent to send logs and metrics, used for the planned log shipping.
hostel-s3-upload (custom)	Allows the server to write to the one static-assets bucket and nothing else. This applies the principle of least privilege.

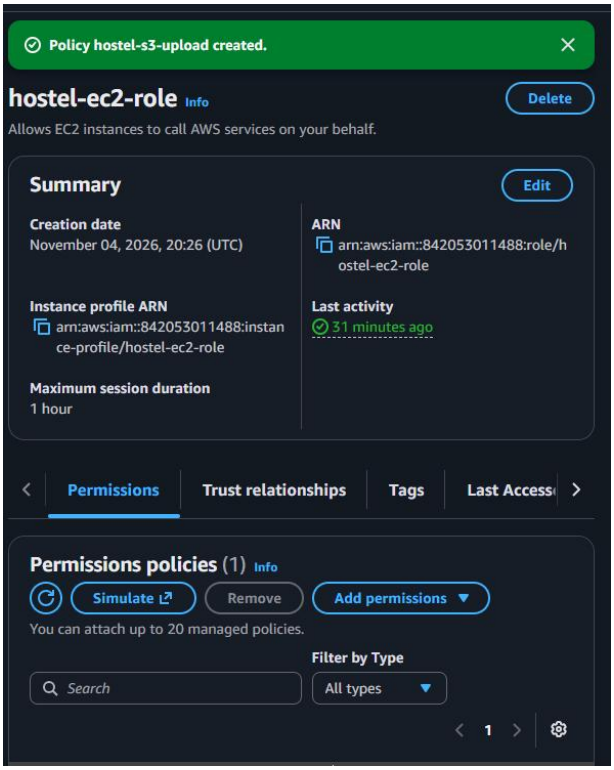


Figure 8. *Custom policy hostel-s3-upload created and attached.*

What Figure 8 shows. The banner confirms that the policy hostel-s3-upload was created. The page shows the role hostel-ec2-role with its creation date (4 November 2026), its ARN, its instance-profile ARN (the object that attaches a role to an EC2 instance) and a maximum session duration of one hour. At this point the role holds one permission policy. The policy text is stored in the repository as deploy/iam/ec2-s3-upload-policy.json.

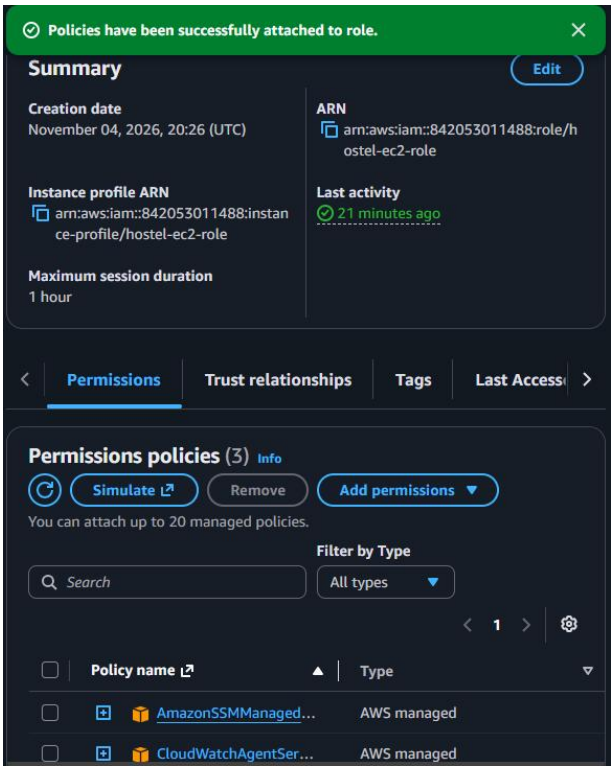


Figure 9. *Managed policies attached to hostel-ec2-role.*

What Figure 9 shows. The banner confirms that policies were attached to the role, and the permissions list now shows three policies, including the two AWS-managed policies

AmazonSSMManagedInstanceCore and CloudWatchAgentServerPolicy. AWS maintains the managed policies, so they stay correct as the services change.

6.3 EC2 instance

The instance is the server that runs the application. Amazon Linux 2023 was chosen because it is maintained by AWS, is optimised for EC2 and uses the dnf package manager. The t3.micro size provides 2 virtual CPUs and 1 GiB of memory, which is sufficient for this application at low cost. It was placed in the default VPC with a public IPv4 address so that it can download packages and reach Supabase, with its security group ensuring that nobody can reach it directly except the administrator and the load balancer.

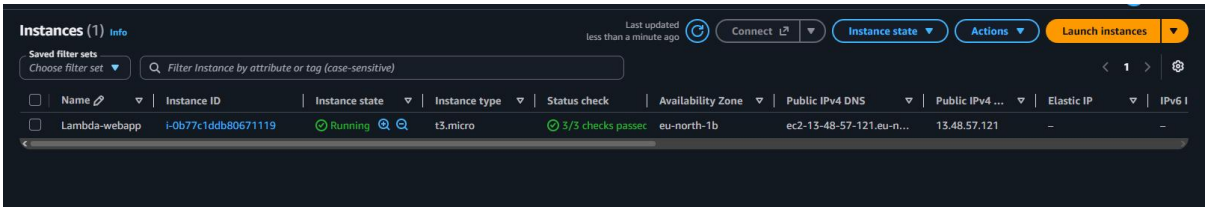


Figure 10. *EC2 instance Lambda-webapp running.*

What Figure 10 shows. The instance list shows one instance named Lambda-webapp. Its state is Running and all 3 of 3 status checks have passed, which means AWS has confirmed that the underlying hardware, the network and the operating system are healthy. The type is t3.micro and the Availability Zone is eu-north-1b. The public IPv4 address and DNS name are listed, and the Elastic IP column is empty, which means the public address will change if the instance is stopped and started. This does not affect users, because they reach the application through CloudFront and the load balancer, never through the instance address.

6.4 Server configuration and build

The administrator connected to the server over SSH using a key pair. The key file must be readable only by its owner (chmod 400), or SSH refuses to use it. After the system was updated, Git and Node.js 20 were installed, the repository was cloned to /home/ec2-user/hostel, the .env file was created on the server and locked with chmod 600, dependencies were installed with npm ci, and the production build was created with npm run build.

```
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ chmod 400 ~/.ssh/Lambda-key3.pem
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ ssh -i ~/.ssh/Lambda-key3.pem ec2-user@13.48.57.121
The authenticity of host '13.48.57.121 (13.48.57.121)' can't be established.
ED25519 key fingerprint is: SHA256:PZ7b8INhoNRMvUu4AnIoXacjDD0CVM8qTUFlXD4W2WI
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.48.57.121' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-40-50 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository                   822 kB/s |  89 kB     00:00
Last metadata expiration check: 0:00:01 ago on Wed Nov  4 20:50:03 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-40-50 ~]$ sudo dnf install -y git
Last metadata expiration check: 0:00:11 ago on Wed Nov  4 20:50:03 2026.
Dependencies resolved.
```

Figure 11. *SSH connection and system preparation.*

What Figure 11 shows. The first two commands set the key permissions and open the SSH session with the key file. On the first connection SSH displays the server's fingerprint and asks for confirmation, which is normal. The Amazon Linux 2023 banner confirms the connection reached the intended server, and the prompt changes to ec2-user@ip-172-31-40-50, showing that commands now run on the AWS instance. The next lines show sudo dnf update -y reporting that nothing needs updating, followed by the installation of Git. The warning about post-quantum key exchange is informational and does not affect the connection.

```
   d5c1ca9..3dcadcb  main -> main
[ec2-user@ip-172-31-40-50 src]$ git pull
Already up to date.
[ec2-user@ip-172-31-40-50 src]$ npm run build

> hostel-visitor-system@1.0.0 build
> next build

▲ Next.js 16.4.4 (Turbopack)
- Environments: .env
⚠ React 18 support is deprecated in Next.js 16 and will be removed in Next.js 17. Please upgrade to React
 19. Learn more: https://nextjs.org/docs/messages/react-version
✓ Running next.config.js took 45ms

⚠ The "middleware" file convention is deprecated. Please use "proxy" instead.

  To migrate automatically, run:
  npx @next/codemod@canary middleware-to-proxy .

  Learn more: https://nextjs.org/docs/messages/middleware-to-proxy
  Creating an optimized production build ...
✓ Compiled successfully in 5.0s
  Running TypeScript ...
  Finished TypeScript in 4.9s ...
  Collecting page data using 1 worker ...
✓ Collecting page data using 1 worker in 846ms
  Generating static pages using 1 worker (0/8) ...
  Generating static pages using 1 worker (2/8)
  Generating static pages using 1 worker (4/8)
  Generating static pages using 1 worker (6/8)
✓ Generating static pages using 1 worker (8/8) in 366ms
```

Figure 12. *Production build of the application.*

What Figure 12 shows. The output shows git pull retrieving the latest commits (including the .npmrc file) and npm run build starting Next.js 16.4.4 with the Turbopack compiler. The line Environments: .env shows that the environment file was found, which is necessary because the public variables are embedded in the build. The two warnings (React 18 support is deprecated in Next.js 16, and the middleware file name is deprecated in favour of proxy) concern future versions and do not stop the build. The build then reports that compilation succeeded in 5.0 seconds, that the TypeScript check completed, and that all 8 static pages were generated.

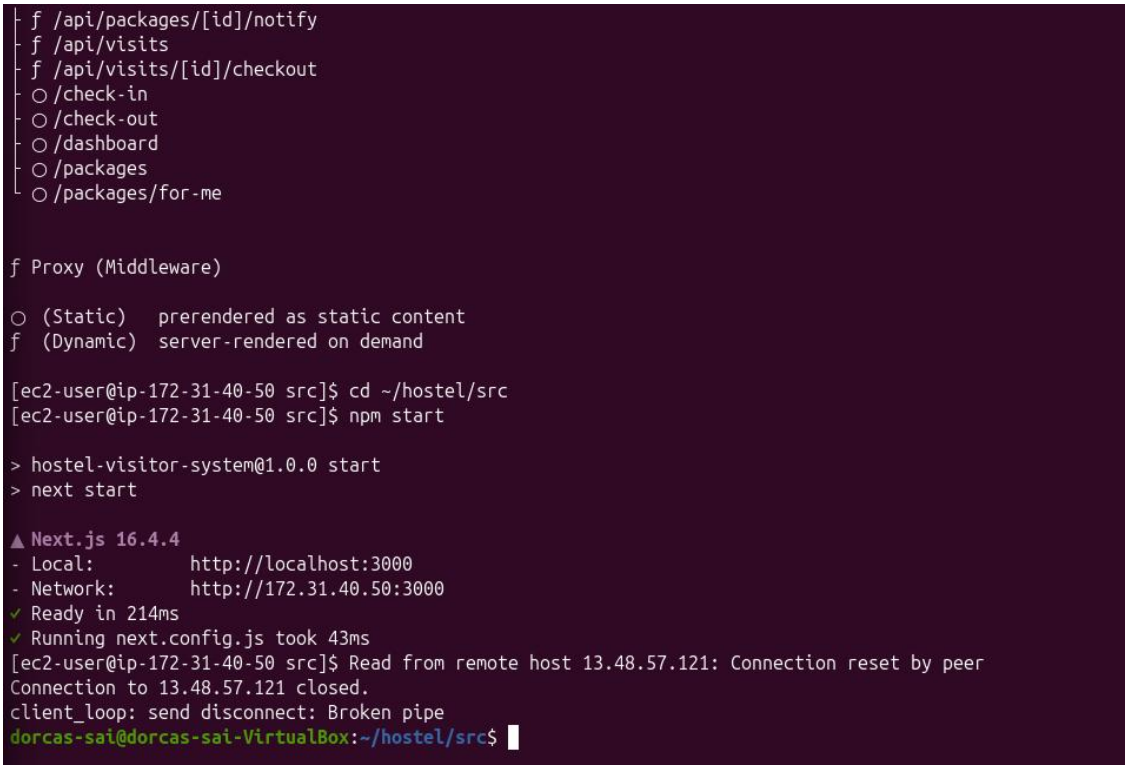


Figure 13. *Route table from the build and the application starting.*

What Figure 13 shows. The route table lists every page and API endpoint. A circle marks a page generated at build time as static content (check-in, check-out, dashboard, packages and packages/for-me), and the letter f marks a route rendered on demand by the server (the API routes for visits, check-out and package notification). The presence of dynamic routes confirms that a server is required. The lower part shows npm start printing that the application is ready in 214 milliseconds and is listening on port 3000. The final lines show the SSH session being closed by the network. This ends only the interactive session; because the application now runs as a service (section 5.5), it is not tied to any login session.

6.5 Running the application as a service

If the application were started by hand, it would stop when the SSH session ended. A systemd service solves this. The unit file hostel.service starts the application as the ec2-user account after the network is available, restarts it automatically if it stops (Restart=always), sets NODE_ENV=production, and, once enabled, starts it again after every reboot. The file is stored in the repository as deploy/hostel.service and listed in Appendix B.

6.6 Target group

The target group is the set of servers to which the load balancer sends traffic. It was configured with the HTTP protocol on port 3000, because that is where Next.js listens, and with a health check on the path

/ . The load balancer requests that path regularly, and only a server that answers successfully receives traffic.

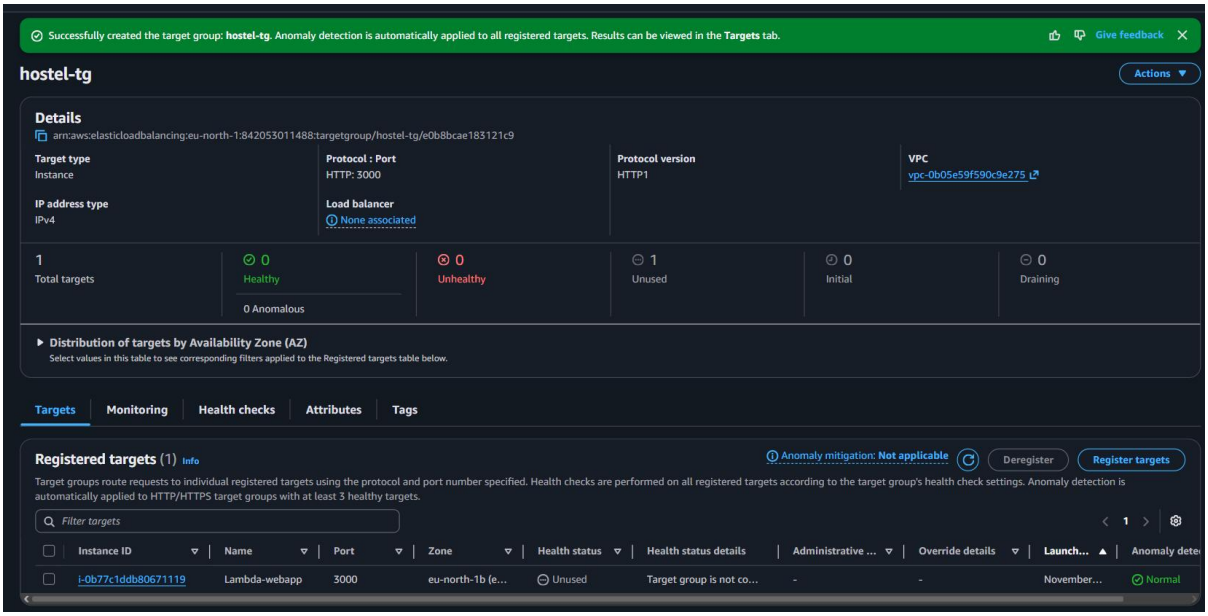


Figure 14. *Target group hostel-tg created with the instance registered.*

What Figure 14 shows. The banner confirms creation of hostel-tg. The details show the target type Instance, the protocol and port HTTP:3000 and the VPC. The Load balancer field reads None associated, because at this moment no listener forwards to the group, so the single registered target is shown as Unused with the explanation that the target group is not configured to receive traffic. This is the expected state before the load balancer exists. The registered-targets table lists the instance Lambda-webapp on port 3000 in zone eu-north-1b, with Anomaly detection reporting Normal.

6.7 Application Load Balancer

An Application Load Balancer gives the application one stable public address, checks the health of the server and routes requests to it. It works at the HTTP level, which allows it to be extended later with more servers or routing rules. It was created as internet-facing and placed in more than one Availability Zone, which AWS requires for an ALB. A listener on HTTP port 80 forwards all requests to hostel-tg.

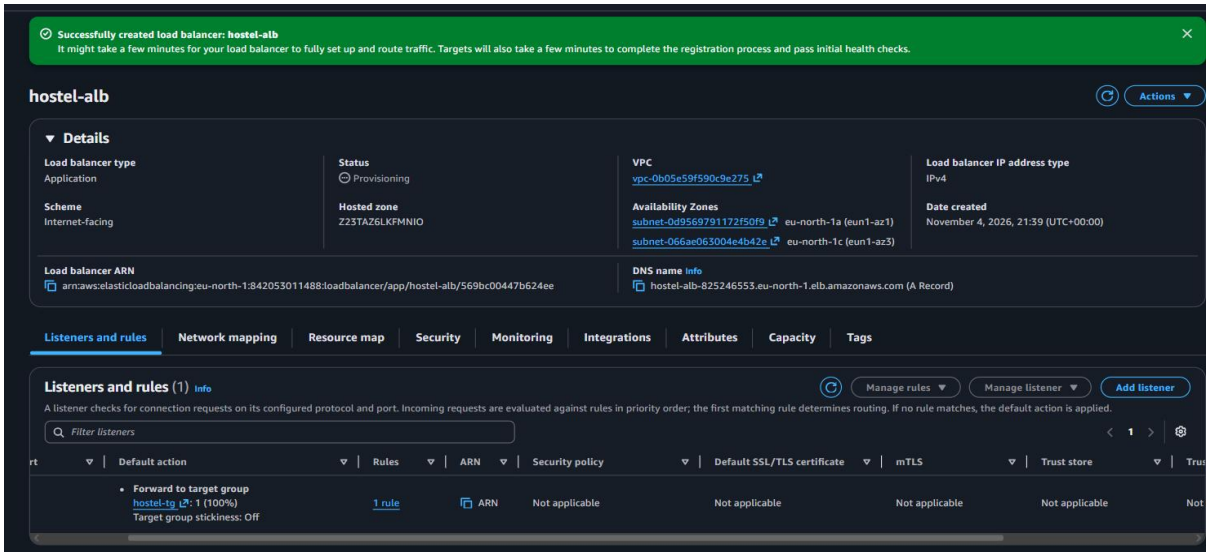


Figure 15. *Application Load Balancer hostel-alb created.*

What Figure 15 shows. The banner confirms the load balancer was created and notes that it can take a few minutes to become active. The details show type Application, scheme Internet-facing, status Provisioning, IPv4 addressing, the VPC, and the Availability Zones selected at creation (eu-north-1a and eu-north-1c). The DNS name is the public address of the load balancer. The Listeners and rules panel shows one rule: the default action forwards 100 percent of traffic to hostel-tg, with stickiness switched off because the application does not need sessions tied to one server. The instance runs in eu-north-1b, which was not selected here; this caused a 503 error that is analysed in section 11 (challenge 8) and was resolved by enabling that zone.

6.8 Functional check through the load balancer

With the zone corrected and the target reporting healthy, the application was opened through the load balancer and used. A test visitor was registered and then appeared on the check-out page. Because that list is read from the database, the page proves the full path: browser, load balancer, EC2 server and Supabase.

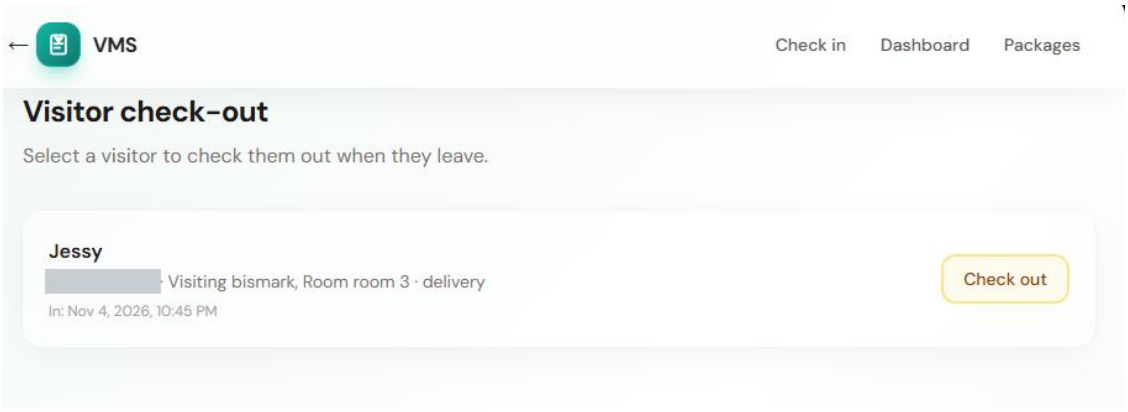


Figure 16. *Visitor check-out page served by the deployed application.*

What Figure 16 shows. The screenshot shows the deployed application with its navigation bar (Check in, Dashboard, Packages). The Visitor check-out page lists one active visitor with the name, the host being visited, the room, the purpose of the visit (delivery) and the check-in time of 4 November 2026, 10:45 PM. A Check out button closes the visit. The visitor's phone number has been hidden to protect personal data.

7. Phase 3: Static content delivery and network lockdown

7.1 S3 bucket for static files

When Next.js builds the application it produces JavaScript and CSS files that never change for a given build. Serving them from the application server would use its limited CPU and memory. Amazon S3 stores them cheaply and durably, and CloudFront delivers them from edge locations. The bucket is private; nothing can read it directly from the internet. CloudFront reads it using Origin Access Control, and the EC2 role may write to it (section 5.2). The files are uploaded to the path _next/static/ because that is the URL path the browser requests. They are uploaded with a long cache lifetime because each build produces files with new names.

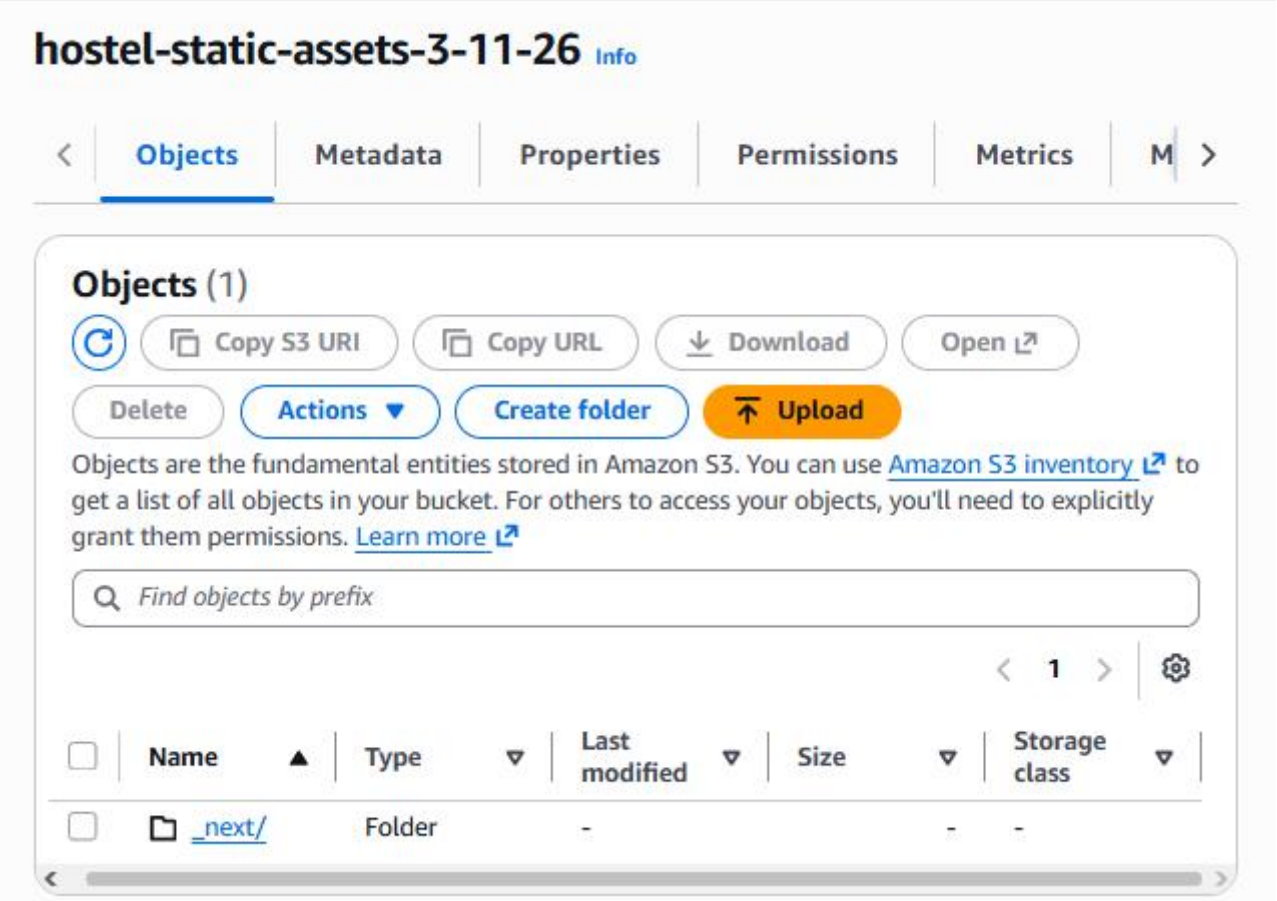


Figure 17. *S3 bucket hostel-static-assets-3-11-26 created.*

What Figure 17 shows. The confirms that the S3 bucket hostel-static-assets-3-11-26 was successfully created. The Objects tab shows the uploaded Next.js static assets inside the next/static folder, confirming that the files were successfully stored in the bucket. The tabs along the top, including Properties, Permissions and Management, provide access to the bucket's configuration, access control and management settings. The date included in the bucket name helps make it unique, as S3 bucket names must be unique across all AWS accounts..

7.2 CloudFront distribution

CloudFront is the public entry point. It was configured with two origins. The Application Load Balancer origin receives pages and API calls through the default behavior, with caching disabled and all HTTP methods allowed, because the application uses POST requests and its pages change with the data. The S3 origin receives requests that match /_next/static/*, which are cached. Viewers who use HTTP are redirected to HTTPS, so every user connection is encrypted.

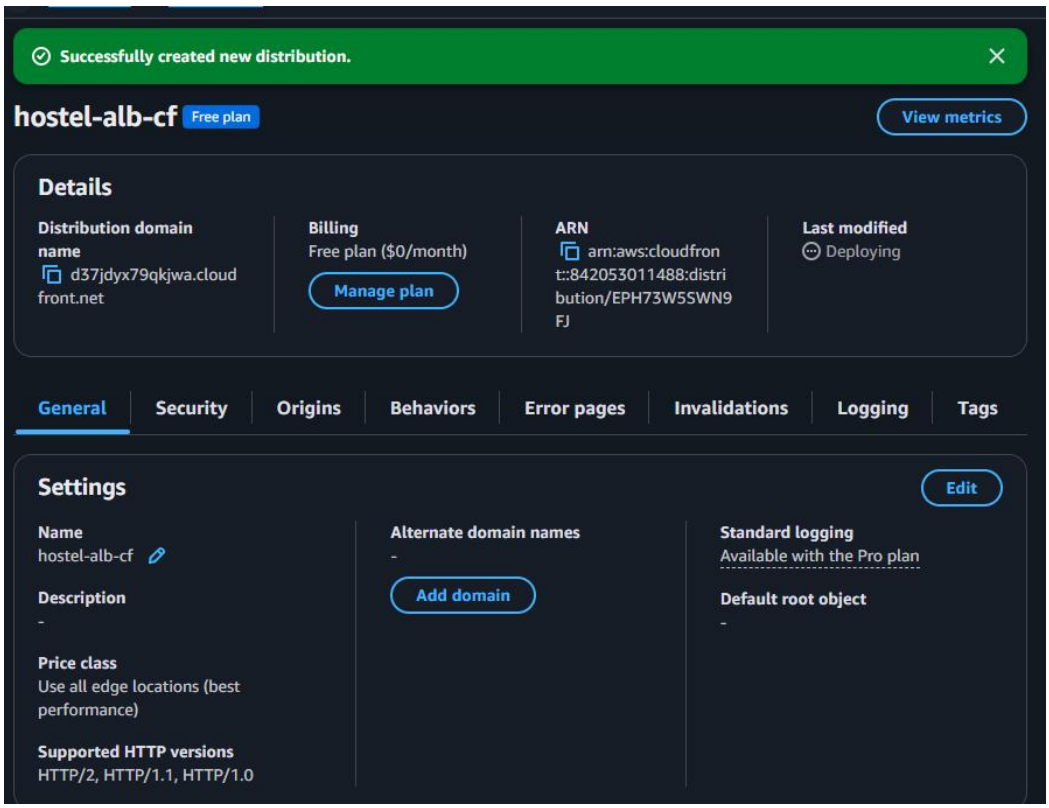


Figure 18. *CloudFront distribution hostel-alb-cf created.*

What Figure 18 shows. The banner confirms the distribution was created. The details show its name hostel-alb-cf, its CloudFront domain name (the public address of the application), the Free plan at no monthly charge, the ARN, and the status Deploying, which means the configuration is being copied to edge locations around the world. Settings show the price class Use all edge locations, support for HTTP/2, HTTP/1.1 and HTTP/1.0, and no alternate domain names, which confirms that the default CloudFront certificate is used.

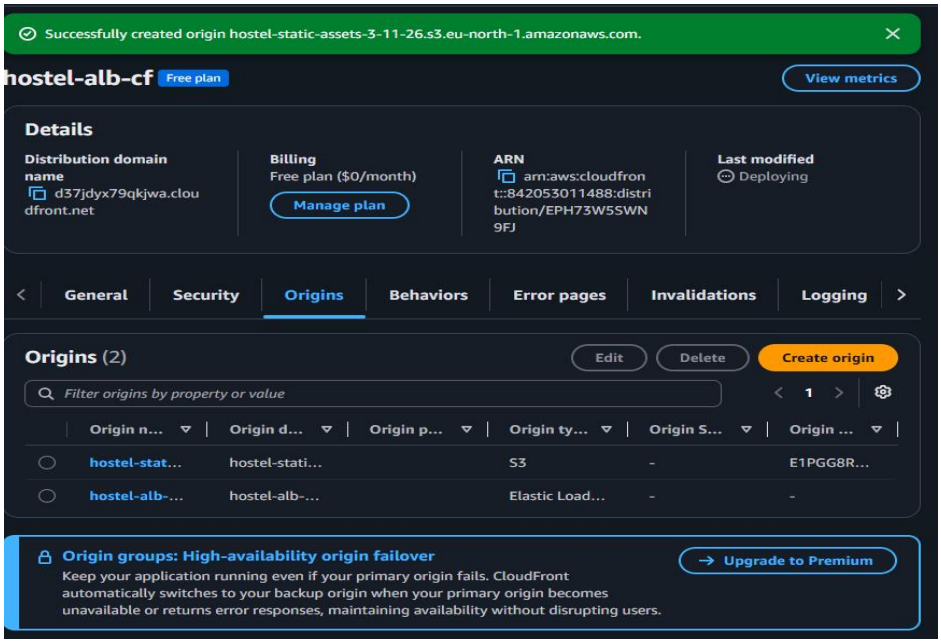


Figure 19. *S3 bucket added as a second origin..*

What Figure 19 shows. The banner confirms that the origin hostel-static-assets-3-11-26.s3.eu-north-1.amazonaws.com was created. The Origins tab now lists two origins: one of type S3 and one of type Elastic Load Balancing. This is the configuration that lets one CloudFront address serve both static files

and dynamic pages. The panel below it, about origin groups with failover, is a paid feature that was not used.

7.3 Locking down the load balancer

Without further action, anyone who discovers the load balancer's own address could bypass CloudFront and its protections. To prevent this, the inbound rule of hostel-alb-sg was changed from any address to the AWS-managed prefix list for CloudFront origin-facing servers. The load balancer now accepts connections only from CloudFront.

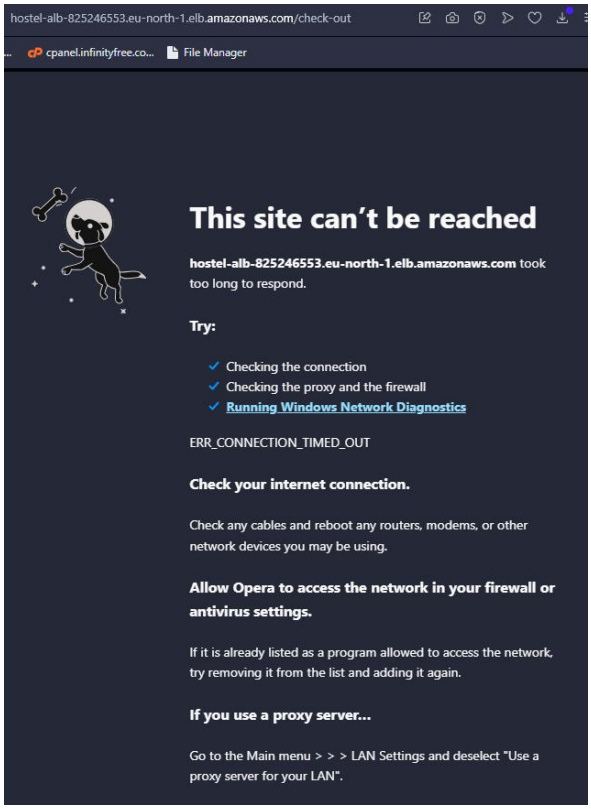


Figure 20. *Direct access to the load balancer address times out.*

What Figure 20 shows. The browser tries to open the load balancer's own address on the path /check-out and reports This site can't be reached, with the error ERR_CONNECTION_TIMED_OUT. In this case the error is the desired result: the security group silently drops traffic that does not come from CloudFront, so the connection never completes. The same application remains reachable through the CloudFront address.

8. Phase 4: Monitoring, cost control and continuous deployment

8.1 Notification channel

Alarms are only useful if someone is told about them. An Amazon SNS topic named hostel-alerts delivers alarm messages by email. SNS requires each email address to confirm its subscription, which prevents anyone from subscribing another person without consent.

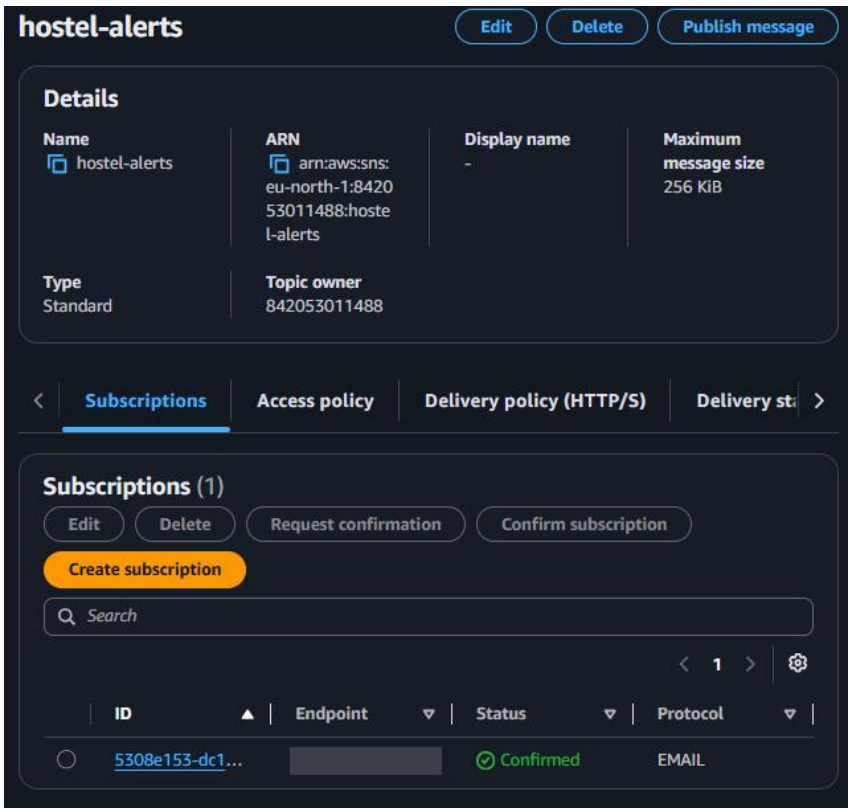


Figure 21. *SNS topic hostel-alerts with a confirmed email subscription.*

What Figure 21 shows. The topic details show the name hostel-alerts, its ARN (which identifies the eu-north-1 region), the type Standard and the account that owns it. The Subscriptions panel lists one subscription with protocol EMAIL and the status Confirmed. A confirmed status means the owner of the address clicked the confirmation link, so alarm messages will be delivered. The email address has been hidden in this copy.

8.2 CloudWatch alarms

Two alarms watch the conditions most likely to indicate a problem. High CPU use suggests the server is overloaded. Server-side errors (HTTP 5xx) reported by the load balancer indicate that users are receiving failures.

Alarm	Condition	Reason
hostel-cpu-high	Average CPU utilisation of the instance above 80 percent over 5 minutes	Sustained high load means the instance is saturated and may need a larger size
hostel-alb-5xx	5 or more HTTP 5xx responses from the load balancer in 5 minutes	Indicates failures that users are experiencing

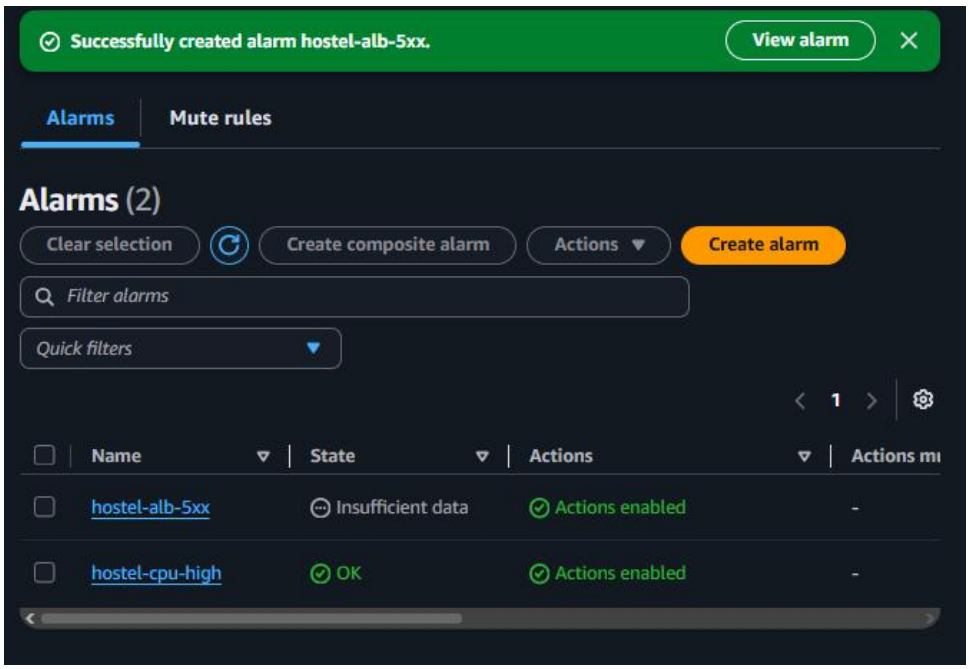


Figure 22. *CloudWatch alarms created.*

What Figure 22 shows. The banner confirms creation of hostel-alb-5xx. The Alarms list shows two alarms with actions enabled. hostel-cpu-high is in the OK state, which means CPU use is below the threshold. hostel-alb-5xx shows Insufficient data, which is normal for a newly created alarm and also when no error responses have been reported, because the metric then has no data points. The state changes to OK once the metric is reported.

8.3 Budget

AWS charges for some resources by the hour, and charges can accumulate unnoticed. A zero-spend budget sends an alert as soon as costs begin, which is the earliest warning AWS Budgets offers.

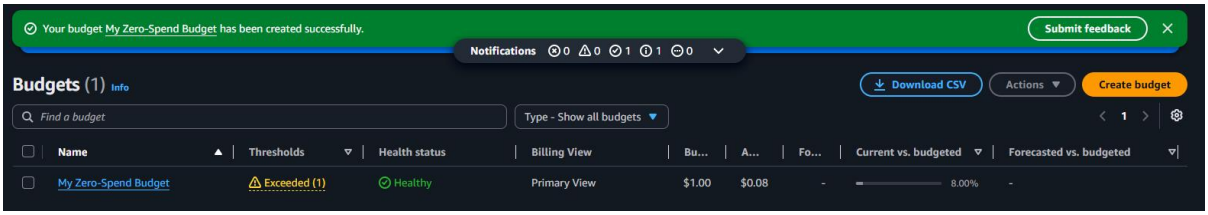


Figure 23. *AWS budget created.*

What Figure 23 shows. The banner confirms creation of My Zero-Spend Budget. The list shows a budget amount of 1.00 USD against actual spending of 0.08 USD, which is 8 percent. The Thresholds column shows Exceeded (1): the zero-spend alert threshold of one cent has been crossed, which is expected now that the instance and load balancer are running and incurring small charges. The Health status Healthy refers to the overall 1.00 USD limit, which has not been reached.This is also to put on record that this account is not a free tier account but a paid one.

8.4 Application logs

The planned design writes the application's output to /var/log/hostel.log and ships it with the CloudWatch agent to a log group named /hostel/app with a retention of 7 days. Central logs let problems be investigated without logging in to the server, and the 7-day retention limits storage cost. The agent configuration is prepared in the repository (deploy/pending/cloudwatch-agent-config.json) and has not yet been applied.

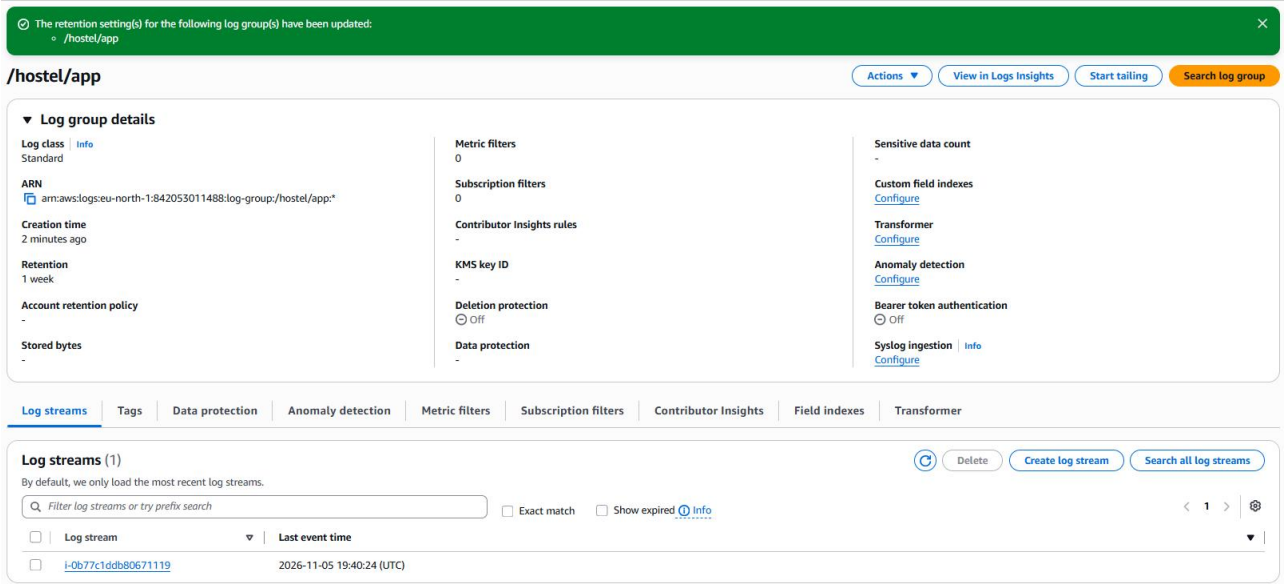


Figure 24. *log group /hostel/app showing retention of 7 days*

8.5 Continuous deployment with GitHub Actions

The aim of this stage is that merging a reviewed pull request into the main branch updates the running application automatically. The workflow (.github/workflows/deploy-ec2.yml) authenticates to AWS using OpenID Connect, so no AWS keys are stored in GitHub, and it runs the update script (deploy/deploy.sh) on the server through AWS Systems Manager, so no SSH port has to be opened to the internet. The workflow has been merged into the main branch through a reviewed pull request. This section remains pending until its first successful run and a before-and-after check on the live site have been recorded.

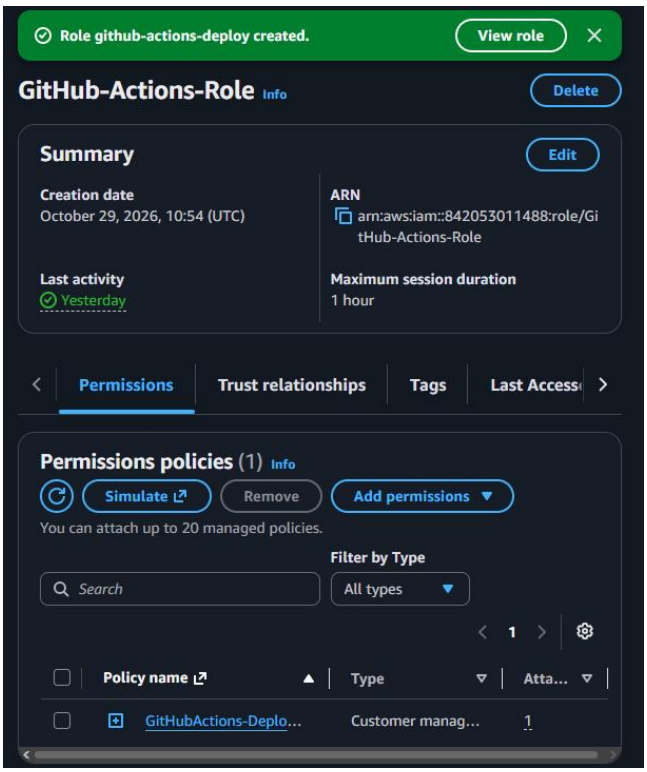


Figure 25. *IAM role created for GitHub Actions.*

What Figure 25 shows. The page shows the role GitHub-Actions-Role, created on 29 October 2026, with a maximum session duration of one hour and one customer-managed permission policy attached. The confirmation banner refers to a second role, github-actions-deploy. Which role the pipeline will use, and whether its trust policy names this repository and the main branch and its permissions allow Systems Manager commands, will be confirmed when the pipeline is finalised.

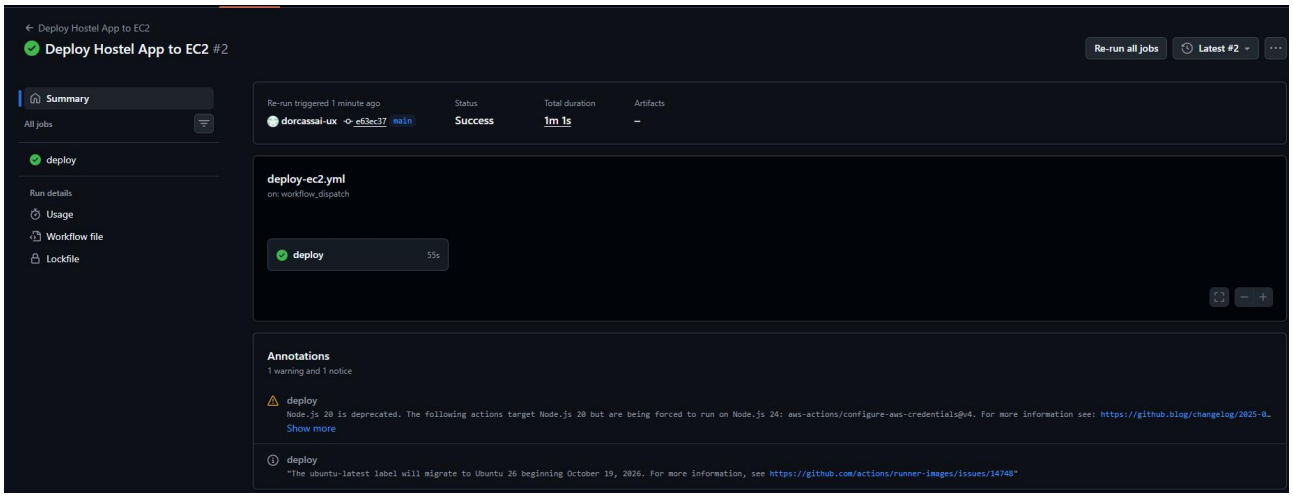


Figure 26. ***Successful GitHub Actions run.***

What Figure 26 shows. The screenshot confirms that the automated deployment workflow completed successfully, with all deployment steps passing in 1 minute and 1 second.

9. Security design

Layer	Control	Threat addressed
Edge	HTTP redirected to HTTPS at CloudFront	Eavesdropping on traffic
Load balancer	Accepts only CloudFront traffic (prefix list); direct access times out	Bypassing CloudFront; direct attacks on the load balancer
Application server	Port 3000 reachable only from the load balancer's security group; SSH only from the administrator's IP	Direct internet access to the server; brute-force SSH attempts
Identity	IAM user for daily work; EC2 role with temporary credentials instead of stored keys	Leaked long-lived credentials
Least privilege	Server may write only to the one static-assets bucket	Damage if the server is compromised
Storage	S3 bucket private; read through CloudFront only	Public exposure of stored files
Secrets	Supabase keys held only in .env on the server (permissions 600); .env excluded from Git	Credential exposure in the repository
Monitoring	CPU and 5xx alarms; budget alert	Undetected failures and cost overruns

Known limitations

- The application has no user login. Anyone who has the address can use it, and its service key gives the server full access to the database. Adding authentication is the most important improvement before real data is entered.
- The application runs on one instance in one Availability Zone, so an instance failure causes downtime.
- No custom domain is configured, so HTTPS uses the default CloudFront certificate.

10. Cost management

The design keeps cost low: a t3.micro instance, one load balancer, a CloudFront plan with no monthly fee, and an S3 bucket holding a few small files. Some resources are charged by the hour whether or not anyone uses the application, notably the load balancer and public IPv4 addresses. The zero-spend budget (Figure 23) gives an immediate warning. After the demonstration and assessment, the resources should be removed in this order:

1. Stop or terminate the EC2 instance.
2. Delete the Application Load Balancer and the target group.
3. Disable and delete the CloudFront distribution, then empty and delete the S3 bucket.
4. Check Billing and Cost Management the following day to confirm that charges have stopped.

11. Verification results

Check	Evidence	Status
EC2 instance running with status checks passed	Figure 10	Verified
Application builds and starts on the server	Figures 12 and 13	Verified
Load balancer created and listener forwards to hostel-tg	Figure 15	Verified
Target reports Healthy	Figure 14	Verified
Application works through the load balancer	Figure 15	Verified
Direct access to the load balancer is blocked	Figure 20	Verified
CloudFront has the load balancer and S3 as origins	Figures 18 and 19	Verified
Static files delivered from S3 through CloudFront	Figure 17	Verified
HTTPS works on the CloudFront address	Figure 18	Verified
SNS subscription confirmed and two alarms created	Figures 21 and 22	Verified
Budget created	Figure 23	Verified
Logs retained for 7 days	Figure 24	Verified
Automatic deployment from GitHub	Figure 25 and 26	Verified

12. Troubleshooting record

Each problem met during the deployment is recorded with its symptom, root cause and resolution, because diagnosing faults is a core part of operating cloud systems.

Challenge 1: Dependency conflict during installation

Root cause. The project depends on eslint-config-next version 16, which requires ESLint 9 or later, while the project pinned an older ESLint. npm refuses to install when peer dependencies conflict, so no packages were installed and the build failed with next: command not found.

Resolution. A .npmrc file containing legacy-peer-deps=true tells npm to relax the peer-dependency check. ESLint is only a code-style checker, so this does not change how the application behaves. The file is committed so that every machine installs the same way.

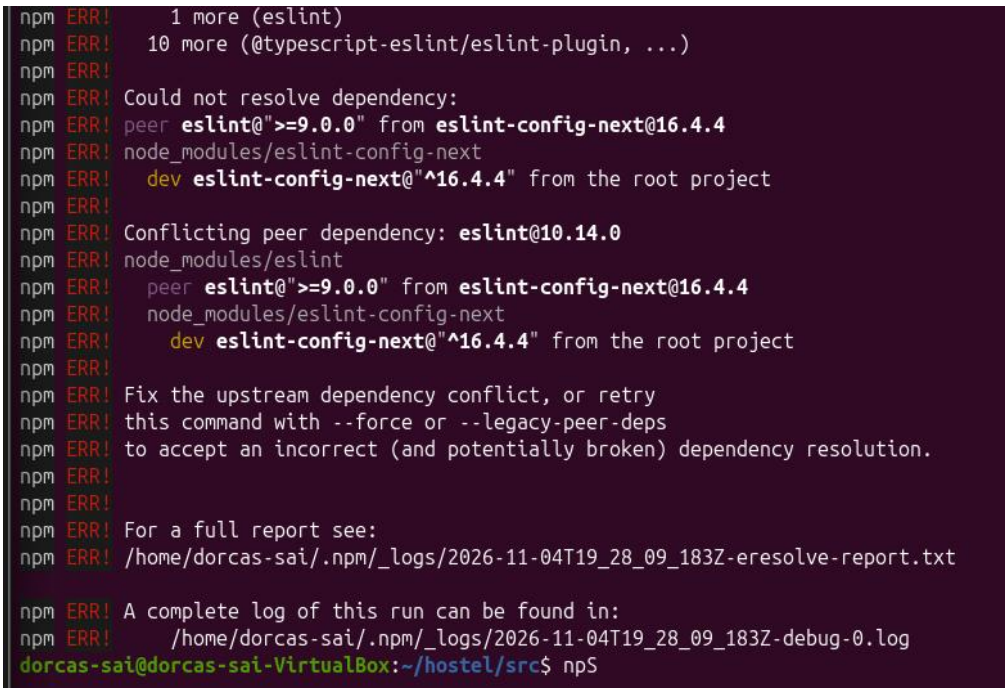


Figure 27. *npm ERESOLVE error.*

What Figure 27 shows. The output states that it could not resolve a dependency: eslint-config-next@16.4.4 requires eslint version 9 or later as a peer, but a conflicting version is present. npm suggests retrying with --force or --legacy-peer-deps. The second option is the safer of the two, and is the one adopted.

Challenge 2: Network time-out while downloading packages

Root cause. The local virtual machine had a slow connection, so a download exceeded npm's default time-out.

Resolution. npm's fetch time-out and retry settings were increased in .npmrc. The installation on the AWS server did not time out, because the server has a fast connection.

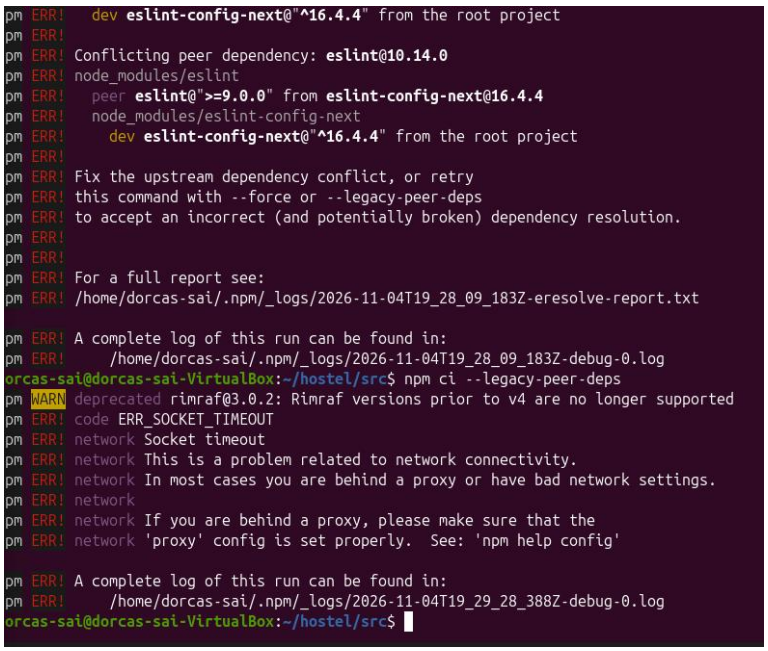


Figure 28. *npm ERR_SOCKET_TIMEOUT.*

What Figure 28 shows. The lower part of the output shows the command npm ci --legacy-peer-deps starting successfully (so the previous conflict is solved) and then failing with code ERR_SOCKET_TIMEOUT, with the explanation that the problem relates to network connectivity.

Challenge 3: Build failed on a TypeScript error

Root cause. In the installed version of Next.js, the params object passed to dynamic API routes (those with [id] in the path) is a Promise, but the code treated it as a plain object.

Resolution. The three affected route handlers (packages notify, packages collect and visits checkout) were changed to declare params as a Promise and to await it. The change was pushed to the repository (Figure 3).

```
.next/types/validator.ts(171,31): error TS2344: Type 'typeof import("/home/dorcas-sai/hostel/src/src/app/
api/packages/[id]/notify/route")' does not satisfy the constraint 'RouteHandlerConfig<"/api/packages/[id]
/notify">'.
  Types of property 'POST' are incompatible.
    Type '(_req: NextRequest, { params }: { params: { id: string; }; }) => Promise<NextResponse<{ error:
string; }> | NextResponse<{ id: string; residentName: string; residentPhone: string; roomNumber: string;
... 7 more ...; isCollected: boolean; }>>' is not assignable to type '(request: NextRequest, context: { p
arams: Promise<{ id: string; }>; }) => void | Response | Promise<void | Response>'.
      Types of parameters '__1' and 'context' are incompatible.
        Type '{ params: Promise<{ id: string; }>; }' is not assignable to type '{ params: { id: string; }
; }'.
          Types of property 'params' are incompatible.
            Property 'id' is missing in type 'Promise<{ id: string; }>' but required in type '{ id: strin
g; }'.
.next/types/validator.ts(191,31): error TS2344: Type 'typeof import("/home/dorcas-sai/hostel/src/src/app/
api/visits/[id]/checkout/route")' does not satisfy the constraint 'RouteHandlerConfig<"/api/visits/[id]/c
heckout">'.
  Types of property 'POST' are incompatible.
    Type '(_req: NextRequest, { params }: { params: { id: string; }; }) => Promise<NextResponse<{ error:
string; }> | NextResponse<{ id: string; visitorName: string; visitorPhone: string; hostName: string; ...
5 more ...; isActive: boolean; }>>' is not assignable to type '(request: NextRequest, context: { params:
Promise<{ id: string; }>; }) => void | Response | Promise<void | Response>'.
      Types of parameters '__1' and 'context' are incompatible.
        Type '{ params: Promise<{ id: string; }>; }' is not assignable to type '{ params: { id: string; }
; }'.
          Types of property 'params' are incompatible.
            Property 'id' is missing in type 'Promise<{ id: string; }>' but required in type '{ id: strin
g; }'.
Failed to type check.

dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ S
```

Figure 29. *TypeScript error in the route handlers.*

What Figure 29 shows. The compiler output reports error TS2344 in the generated validator file, explaining that the POST handler is not assignable because the type of params is incompatible: a Promise of an object is expected where a plain object was supplied. The build ends with Failed to type check.

Challenge 4: Push to the team repository rejected (403)

Root cause. Authentication succeeded, but the account did not yet have permission to write to a repository owned by another account.

Resolution. Once write access to the team repository was in place, the push completed (Figure 3).

```
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ git commit -m "fix: await route params for newer Next.js"
[main d5c1ca9] fix: await route params for newer Next.js
 5 files changed, 12 insertions(+), 8 deletions(-)
 create mode 100644 src/.npmrc
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ git push
Username for 'https://github.com': ^C
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$ git push
Username for 'https://github.com': dorcassai-ux
Password for 'https://dorcassai-ux@github.com':
remote: Permission to Ewura-bena685/Lambda_Capstone_aws-secure-web-app.git denied to dorcassai-ux.
fatal: unable to access 'https://github.com/Ewura-bena685/Lambda_Capstone_aws-secure-web-app.git/': The r
equested URL returned error: 403
dorcas-sai@dorcas-sai-VirtualBox:~/hostel/src$
```

Figure 30. *HTTP 403 on git push.*

What Figure 30 shows. The upper part shows the commit being created with 5 changed files, including the new .npmrc. The lower part shows git asking for credentials and then the message that permission to the team repository was denied to the account, with HTTP error 403. A 403 means the identity is known but not allowed, which differs from a 401 where the identity itself is rejected.

Challenge 5: EC2 console Connect button failed

Root cause. The browser-based Connect feature reaches the instance from AWS-owned addresses, but the SSH rule allows only the administrator's IP address.

Resolution. This is the security group working as intended, and the rule was not loosened. SSH from the administrator's own machine was used instead.

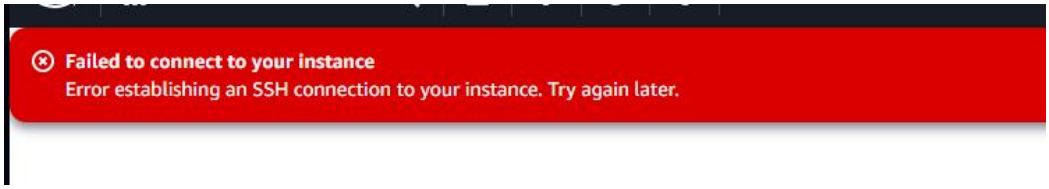


Figure 31. *EC2 Instance Connect failure.*

What Figure 31 shows. The red banner reads Failed to connect to your instance: Error establishing an SSH connection. It confirms that SSH access is blocked for every source except the allowed address.

Challenge 6: Security group rule could not be edited

Root cause. A rule that was created with another security group as its source cannot be converted into a rule that uses an IP address.

Resolution. The old rule was deleted and a new SSH rule was created with the source set to the administrator's IP (My IP).

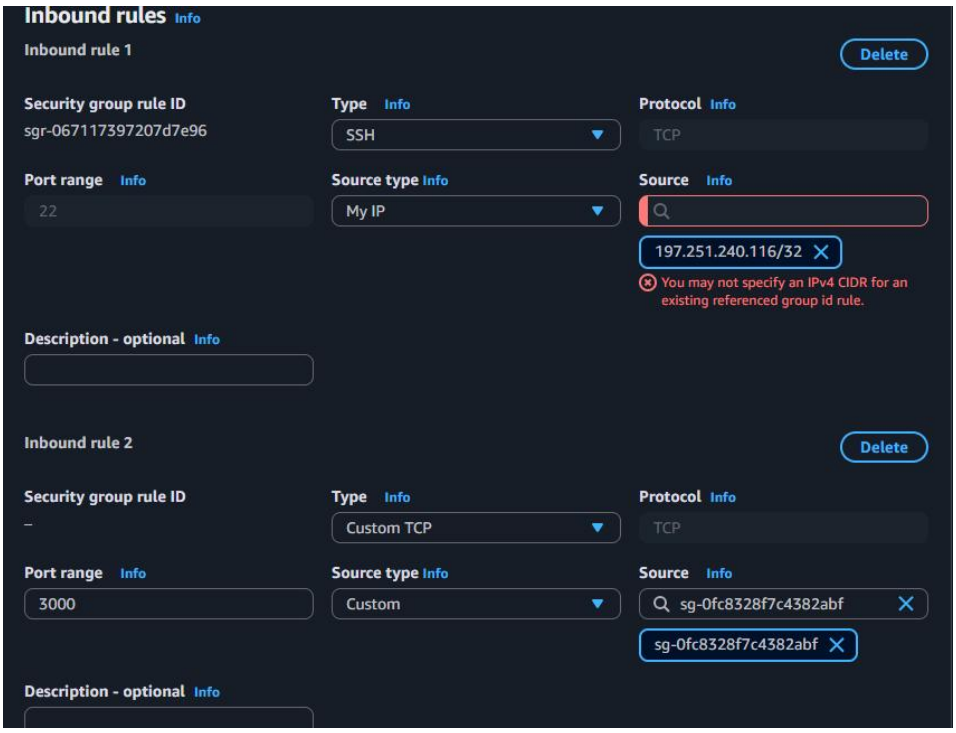


Figure 32. *Inbound rules edit screen with an error.*

What Figure 32 shows. Inbound rule 1 (SSH, port 22, source type My IP) shows the error You may not specify an IPv4 CIDR for an existing referenced group id rule. Inbound rule 2 (Custom TCP, port 3000) correctly uses the security group sg-0fc8328f7c4382abf, the load balancer's group, as its source.

Challenge 7: Service failed to start because the port was in use

Root cause. A copy of the application started by hand during testing was still running and held port 3000, so the systemd service could not bind to it.

Resolution. The leftover process was stopped, the failed state was cleared with systemctl reset-failed, and the service was started again. For future testing, the service is stopped first.

```
}
[ec2-user@ip-172-31-40-50 src]$ sudo ss -ltnp | grep 3000
LISTEN 0      511                *:3000            *:*    users:(("next-server (v1",pid=33046,fd=20))
[ec2-user@ip-172-31-40-50 src]$ pkill -f "next start"
[ec2-user@ip-172-31-40-50 src]$ npm start

> hostel-visitor-system@1.0.0 start
> next start

⨯ Failed to start server
Error: listen EADDRINUSE: address already in use :::3000
    at new Promise (<anonymous>) {
  code: 'EADDRINUSE',
  errno: -98,
  syscall: 'listen',
  address: '::',
  port: 3000
}
[ec2-user@ip-172-31-40-50 src]$
```

Figure 33. *Port 3000 already in use.*

What Figure 33 shows. The first command, ss -ltnp, shows a process called next-server with process number 33046 listening on port 3000. The pkill command used the wrong name pattern and had no effect. Starting the application then fails with Error: listen EADDRINUSE: address already in use :::3000, which identifies the cause precisely.

Challenge 8: Load balancer returned 503

Root cause. The instance is in the Availability Zone eu-north-1b, but the load balancer was created with only eu-north-1a and eu-north-1c enabled. A load balancer can send traffic only to targets in zones it has enabled, so no target was available.

Resolution. The zone eu-north-1b was enabled in the load balancer's network mapping. The target changed to Healthy and the application loaded.



Figure 34. *503 Service Temporarily Unavailable.*

What Figure 34 shows. The browser shows the load balancer's address and the message 503 Service Temporarily Unavailable. A response generated by the load balancer itself (not by the application) means the load balancer is working but has no healthy target to forward to.

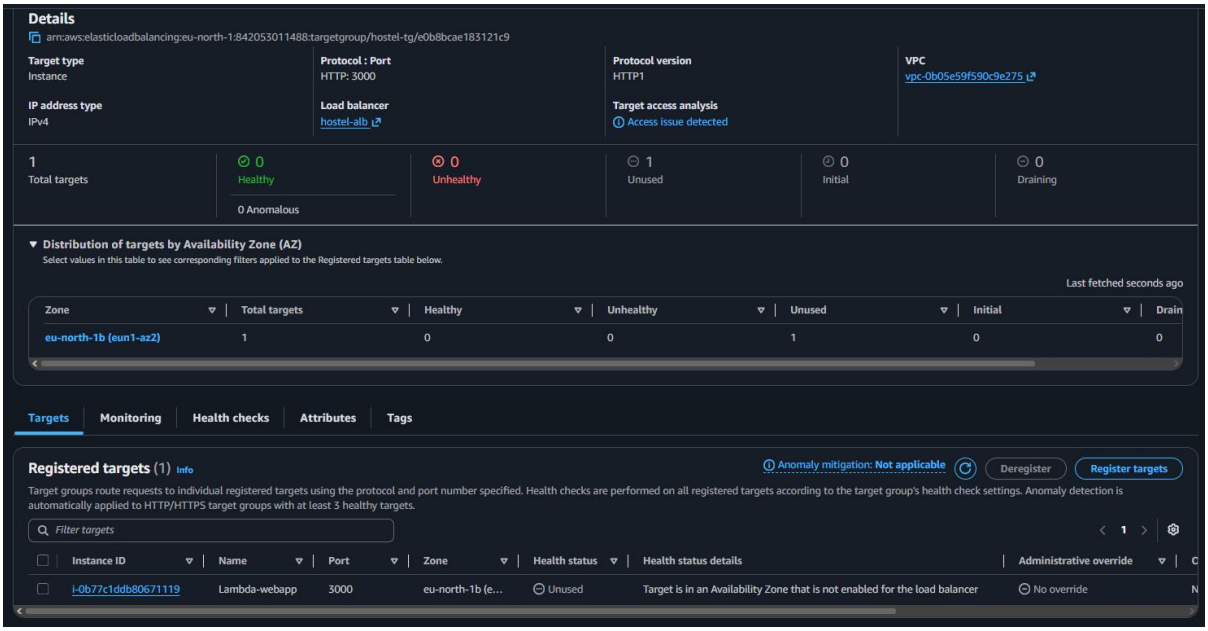


Figure 35. *Target group showing the Availability Zone problem.*

What Figure 35 shows. The summary shows 1 target in the Unused state and a Target access analysis note reading Access issue detected. The registered target Lambda-webapp, on port 3000 in zone eu-north-1b, has the health status Unused and the details Target is in an Availability Zone that is not enabled for the load balancer. This message names the exact cause and fix.

13. Lessons Learned

1. **Understand the application before designing the infrastructure.** The app has server-side API routes, so it could not be hosted as static files on S3. It needed a server, which shaped the whole architecture.
2. **Build in layers and test each one.** The app was tested locally, then on the server, then behind the load balancer, then behind CloudFront. When something failed, the fault was in the newest layer, and the error messages named the cause precisely: the zone not enabled for the load balancer (503), the port already in use, and the permission denied on push.
3. **Apply least privilege everywhere.** Only the load balancer can reach the app, only CloudFront can reach the load balancer, the server uses a role with no stored keys, and GitHub signs in with

OpenID Connect instead of access keys. A blocked connection is often the security design working as intended, as with the console Connect button failing.

4. **Size the server for the job, and automate the repeated work.** A 1 GB instance can run the app but struggles to build it, and running two builds at once made it worse. A systemd service, an update script and a GitHub Actions workflow replaced manual steps that were easy to get wrong.
5. **Plan for cost, change and collaboration.** The load balancer and public IP are charged by the hour, and without an Elastic IP the server's address changes after every stop and start. On a shared repository, check permissions and review changes through pull requests. Record screenshots as you work, with private details hidden.

14. Repository contents

All code and configuration used in this deployment is stored in the GitHub repository, so that the environment can be rebuilt by anyone on the team. Secrets are not stored. The table lists each item and its purpose.

Path in repository	Purpose
src/ (existing)	npm settings: legacy-peer-deps and network time-outs (challenges 1 and 2)
src/.npmrc	Project README: features, setup, deployment, verification, troubleshooting
README.md	Architecture diagram used in the README and this document
docs/architecture.png	Technical documentation in Word and PDF format
docs/ (this document)	systemd unit that runs the application
deploy/hostel.service	Update script: pull, build, upload static files, restart the service
deploy/deploy.sh	Custom IAM policy hostel-s3-upload
deploy/iam/ec2-s3-upload-policy.json	Inbound rules of the two security groups
deploy/security-group-rules.md	Explains each deployment file and where it is used
deploy/README.md	GitHub Actions workflow that deploys the application through Systems Manager (section 7.5)
.github/workflows/deploy-ec2.yml	Permission and trust policies for the GitHub Actions role
deploy/iam/github-actions-*.json	CloudWatch agent configuration for log shipping
deploy/pending/cloudwatch-agent-config.json	Amazon Machine Image. The template an EC2 server starts from.

The deployment workflow runs only when files under src/ change on the main branch, or when it is started manually, so editing documents or deployment files does not trigger a release.

Planned improvements: user authentication, a custom domain with an ACM certificate, and two instances in an Auto Scaling group across Availability Zones.

Appendix A: Resource inventory

Service	Name	Key settings
EC2	Lambda-webapp	t3.micro, Amazon Linux 2023, eu-north-1b
Security group	hostel-alb-sg	Inbound HTTP 80 from the CloudFront prefix list
Security group	hostel-ec2-sg	TCP 3000 from hostel-alb-sg; SSH 22 from the administrator's IP
Application Load Balancer	hostel-alb	Internet-facing, IPv4, listener HTTP 80 forwarding to hostel-tg
Target group	hostel-tg	HTTP, port 3000, health check path /
S3 bucket	hostel-static-assets-3-11-26	eu-north-1, private
CloudFront distribution	hostel-alb-cf	Origins: load balancer and S3; HTTP redirected to HTTPS
IAM role	hostel-ec2-role	AmazonSSMManagedInstanceCore, CloudWatchAgentServerPolicy, hostel-s3-upload
IAM role	GitHub-Actions-Role	For the CI/CD pipeline
SNS topic	hostel-alerts	Email subscription confirmed
CloudWatch alarms	hostel-cpu-high, hostel-alb-5xx	CPU above 80 percent; 5 or more 5xx responses in 5 minutes
Budget	My Zero-Spend Budget	Alert at the first cost

Appendix B: Configuration listings

B.1 systemd unit (deploy/hostel.service)

```
[Unit]
Description=Hostel Visitor Management System
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/hostel/src
ExecStart=/usr/bin/npm start
Restart=always
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

B.2 npm settings (src/.npmrc)

```
fetch-timeout=600000
fetch-retries=5
fetch-retry-mintimeout=20000
```

```
fetch-retry-maxtimeout=120000
legacy-peer-deps=true
```

B.3 Update script (deploy/deploy.sh)

```
#!/bin/bash
set -euo pipefail

REPO_DIR="/home/ec2-user/hostel"
APP_DIR="${REPO_DIR}/src"
BUCKET="hostel-static-assets-3-11-26"
REGION="eu-north-1"

cd "$REPO_DIR"
git fetch origin main
git reset --hard origin/main

cd "$APP_DIR"
npm ci
npm run build

aws s3 sync .next/static "s3://${BUCKET}/_next/static" \
  --region "$REGION" \
  --cache-control "public,max-age=31536000,immutable"

sudo systemctl restart hostel
echo "Deploy finished"
```

B.4 Custom IAM policy (deploy/iam/ec2-s3-upload-policy.json)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::hostel-static-assets-3-11-26/*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::hostel-static-assets-3-11-26"
    }
  ]
}
```

B.5 Environment variables (names only)

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
```

The values are held only in the .env file on the server and are never committed to the repository.